Relational Algebra Reference

Database Systems - Lecture 2

Your Comprehensive Guide to Solving Relational Algebra Problems

January 8, 2026

Contents

1 Relational Algebra: The Foundation of Database Queries **2**

2 Relational Model Basics **2**
- 2.1 Core Terminology . . . 2
- 2.2 Keys . . . 3

3 The Six Basic Operations **4**
- 3.1 SELECT (σ) - Filter Rows . . . 4
- 3.2 PROJECT (Π) - Select Columns . . . 6
- 3.3 Combining SELECT and PROJECT . . . 8
- 3.4 CARTESIAN PRODUCT ($\times$) - Combine Every Row . . . 9
- 3.5 JOIN ($\bowtie$) - Smart Combination . . . 11
  - 3.5.1 Theta Join . . . 11
  - 3.5.2 Natural Join . . . 12
- 3.6 UNION ($\cup$) - Combine Rows from Compatible Tables . . . 14
- 3.7 INTERSECTION ($\cap$) - Common Rows . . . 15
- 3.8 SET DIFFERENCE ($-$) - Rows in One But Not Other . . . 15

4 Advanced Operations **17**
- 4.1 RENAME (ρ) - Change Names . . . 17
- 4.2 ASSIGNMENT ($\leftarrow$) - Store Intermediate Results . . . 17
- 4.3 Outer Joins - Keeping Non-Matching Rows . . . 19

5 Aggregation and Grouping **22**
- 5.1 Example 1: Count Students per Semester . . . 23
- 5.2 Example 2: Multiple Aggregate Functions . . . 24
- 5.3 Example 3: All Aggregate Functions . . . 24
- 5.4 Aggregation Without Grouping . . . 26
- 5.5 Combining with Other Operations . . . 26

6 Operation Commutativity and Ordering **28**
- 6.1 Commutative Operations . . . 28
- 6.2 NON-Commutative Operations . . . 28

7 Query Optimization Rules **30**
7.1 Rule 1: Push SELECT Down . . . 30
7.2 Rule 2: Push PROJECT Down . . . 30
7.3 Rule 3: Combine Multiple SELECTs . . . 31
7.4 Rule 4: Use Smaller Relation First in Join . . . 31

8 Common Query Patterns and Examples **32**
8.1 Pattern 1: Filter and Project . . . 32
8.2 Pattern 2: Join Two Tables . . . 32
8.3 Pattern 3: Join Multiple Tables . . . 32
8.4 Pattern 4: Union - Combine Results . . . 33
8.5 Pattern 5: Intersection - Find Common . . . 33
8.6 Pattern 6: Difference - Find Missing . . . 33

9 Complete Worked Examples **35**
9.1 Example 1: Simple Query . . . 35
9.2 Example 2: Join Query . . . 36
9.3 Example 3: Set Operations . . . 38

10 Common Mistakes and How to Avoid Them **39**
10.1 Mistake 1: Wrong Order of Operations . . . 39
10.2 Mistake 2: Forgetting Duplicates in Project . . . 39
10.3 Mistake 3: Confusing Difference Order . . . 39
10.4 Mistake 4: Natural Join with Wrong Attributes . . . 39
10.5 Mistake 5: Forgetting Union-Compatibility . . . 40

11 Quick Reference: All Operations **41**

12 Tips for Solving Relational Algebra Problems **42**

13 Practice Problems **44**
13.1 Problem Set . . . 44
13.2 Solutions . . . 45

14 Final Tips for Success **47**

15 Gør vi det **48**

1 Relational Algebra: The Foundation of Database Queries

Why Relational Algebra Matters

What is it?
Relational Algebra is the theoretical foundation for SQL and databases. It's a collection of operations that let you manipulate and query data stored in tables (relations).

Why should you care?

- SQL is based on relational algebra
- Understanding RA helps you write better queries
- Query optimizers use RA to make queries faster
- It's the mathematical language of databases

The Core Idea:
Think of relations as tables. Relational algebra gives you operations to:

- Filter rows (SELECT: σ)
- Choose columns (PROJECT: Π)
- Combine tables (JOIN: $\bowtie$, UNION: $\cup$, etc.)
- And much more!

2 Relational Model Basics

2.1 Core Terminology

Relation

Relation: A table (set of tuples with the same structure)
Tuple: A row in the table
Attribute: A column in the table

Example:

ID	Name	Dept	Salary
101	Alice	CS	5000
102	Bob	Math	6000
103	Carol	CS	5500

- **Relation:** The whole table (call it "Instructor")
- **Tuple:** One row, e.g., $(101, \text{Alice}, \text{CS}, 5000)$
- **Attributes:** ID, Name, Dept, Salary

Key Properties of Relations

Important Properties:

1. **Order of tuples doesn't matter** - Relations are sets, not lists
2. **No duplicate tuples** - Each tuple is unique
3. **Attributes are atomic** - No nested structures in a single cell
4. **Domain:** Each attribute has a type (integers, strings, etc.)

2.2 Keys

Types of Keys

Superkey: A set of attributes that uniquely identifies each tuple
Candidate Key: A minimal superkey (no subset is also a superkey)
Primary Key: The chosen candidate key (underlined in schema)
Foreign Key: An attribute that references the primary key of another relation

Example Schema:
`Instructor(ID, Name, Dept, Salary)`
`Department(Dept, Building, Budget)`
Here:

- ID is primary key of Instructor
- Dept is primary key of Department
- Dept in Instructor is a foreign key referencing Department

3 The Six Basic Operations

3.1 SELECT (σ) - Filter Rows

SELECT Operation

Notation: $\sigma_{\text{condition}}(R)$
What it does: Keeps only rows that satisfy the condition
Think of it as: "Give me rows WHERE condition is true"

Conditions can use: $=$, $\neq$, $<$, $>$, $\leq$, $\geq$, $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)

SELECT Examples

Input Table: Student

ID	Name	Age
1	Alice	20
2	Bob	22
3	Carol	20
4	Dave	23

Example 1: $\sigma_{\text{Age}=20}(\text{Student})$
Result: Students who are 20 years old

ID	Name	Age
1	Alice	20
3	Carol	20

Example 2: $\sigma_{\text{Age}>21}(\text{Student})$
Result: Students older than 21

ID	Name	Age
2	Bob	22
4	Dave	23

Example 3: $\sigma_{\text{Age}=20\wedge\text{Name}=\text{'Alice'}}(\text{Student})$
Result: Students who are 20 AND named Alice

ID	Name	Age
1	Alice	20

Key Insights About SELECT

- **Horizontal filtering** - Works on rows
- **Unary operation** - Takes ONE relation as input
- **Output schema = Input schema** - Same columns
- **Number of rows** - Can only decrease or stay same

- **Commutative:** $\sigma_p(\sigma_q(R)) = \sigma_q(\sigma_p(R)) = \sigma_{p \wedge q}(R)$

3.2 PROJECT (Π) - Select Columns

PROJECT Operation

Notation: $\Pi_{A_1,A_2,\ldots,A_k}(R)$
What it does: Keeps only specified columns, removes duplicates
Think of it as: "Show me only these columns"

Important: Duplicate rows are automatically removed!

PROJECT Examples

Input Table: Student

ID	Name	Age
1	Alice	20
2	Bob	22
3	Carol	20
4	Dave	23

Example 1: $\Pi_{\text{Name}}(\text{Student})$
Result: Just the names

Name
Alice
Bob
Carol
Dave

Example 2: $\Pi_{\text{Age}}(\text{Student})$
Result: Ages (duplicates removed!)

Age
20
22
23

Note: Age 20 appears only once even though 2 students are 20!

Example 3: $\Pi_{\text{Name, Age}}(\text{Student})$

Name	Age
Alice	20
Bob	22
Carol	20
Dave	23

No duplicates here since Name+Age combinations are unique!

Key Insights About PROJECT

- **Vertical filtering** - Works on columns
- **Unary operation** - Takes ONE relation as input
- **Removes duplicates automatically** - Output is a set
- **Number of columns decreases or stays same**
- **NOT always commutative:** $\Pi_A(\Pi_B(R))$ only makes sense if $A \subseteq B$

3.3 Combining SELECT and PROJECT

Order Matters!

Key Question: Does order matter when combining σ and Π?

Answer: YES! You must ensure needed attributes exist!

Rule of Thumb:

- Do SELECT first if you need attributes for filtering
- Then PROJECT to get final columns

Combining Operations - Correct vs Wrong

Goal: Find names of students older than 21
Input Table: Student

ID	Name	Age
1	Alice	20
2	Bob	22
3	Carol	20
4	Dave	23

CORRECT: $\Pi_{\text{Name}}(\sigma_{\text{Age}>21}(\text{Student}))$
Step 1: $\sigma_{\text{Age}>21}(\text{Student})$ gives:

ID	Name	Age
2	Bob	22
4	Dave	23

Step 2: Π_{Name} gives:

Name
Bob
Dave

WRONG: $\sigma_{\text{Age}>21}(\Pi_{\text{Name}}(\text{Student}))$
Step 1: $\Pi_{\text{Name}}(\text{Student})$ gives:

Name
Alice
Bob
Carol
Dave

Step 2: $\sigma_{\text{Age}>21}$ **FAILS!** - Age column doesn't exist anymore!

3.4 CARTESIAN PRODUCT (×) - Combine Every Row

Cartesian Product

Notation: $R \times S$
What it does: Combines EVERY row from R with EVERY row from S
Think of it as: "All possible pairings"

Result size: If R has n rows and S has m rows, result has $n \times m$ rows
Result columns: All columns from R + all columns from S

Cartesian Product Example

Table A: Student

Name	Age
Alice	20
Bob	22

Table B: Course

CID	Title
CS101	Databases
CS102	Algorithms

Result: Student × Course

Name	Age	CID	Title
Alice	20	CS101	Databases
Alice	20	CS102	Algorithms
Bob	22	CS101	Databases
Bob	22	CS102	Algorithms

$2 \times 2 = 4$ rows in result!

Cartesian Product Warning

DANGER: Cartesian product creates HUGE tables!

- 100 rows × 100 rows = 10,000 rows!
- 1000 rows × 1000 rows = 1,000,000 rows!

When to use:

- Almost NEVER by itself
- Usually combined with SELECT to filter
- Better to use JOIN operations (coming next!)

Key Insights About Cartesian Product

- **Binary operation** - Takes TWO relations
- **Commutative:** $R \times S = S \times R$ (same data, different column order)
- **Associative:** $(R \times S) \times T = R \times (S \times T)$
- **Rarely used alone** - Usually with SELECT to filter

3.5 JOIN ($\bowtie$) - Smart Combination

Join Operations

General Idea: Combine tables based on a condition

Types of Joins:

1. **Theta Join:** $R \bowtie_\theta S$ - Join with any condition θ
2. **Natural Join:** $R \bowtie S$ - Join on common attributes (equal values)

Key Relationship:

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

Join = Cartesian Product + SELECT!

3.5.1 Theta Join

Theta Join Example

Table: Student

SID	Name	Age
1	Alice	20
2	Bob	22

Table: Enrollment

SID	Course
1	CS101
2	CS102

Query: Student $\bowtie_{\text{Student.SID=Enrollment.SID}}$ Enrollment
Result:

Student.SID	Name	Age	Enrollment.SID	Course
1	Alice	20	1	CS101
2	Bob	22	2	CS102

How it works:

1. Create all combinations (Cartesian product)
2. Keep only rows where Student.SID = Enrollment.SID

3.5.2 Natural Join

Natural Join

Notation: $R \bowtie S$
What it does:

1. Finds common attribute names in R and S
2. Joins on equality of those attributes
3. Removes duplicate columns in result

Key Difference from Theta Join:

- Automatically finds join columns
- Each common attribute appears ONCE in result

Natural Join Example

Table: Student

SID	Name	Age
1	Alice	20
2	Bob	22

Table: Enrollment

SID	Course
1	CS101
2	CS102

Query: Student $\bowtie$ Enrollment
Result:

SID	Name	Age	Course
1	Alice	20	CS101
2	Bob	22	CS102

Note: SID appears only ONCE! (Compare to theta join)

Natural Join Pitfalls

Be Careful! Natural join can be dangerous:

- Joins on ALL common attributes (might not be what you want)
- If no common attributes: gives Cartesian product!
- If many common attributes: might over-constrain

Example of Danger:

Suppose both Student and Takes tables have a "Name" attribute:
Student:

SID	Name	Major
1	Alice	CS
2	Bob	Math
3	Alice	Physics

Takes:

SID	Name	Course
1	Alice	CS101
2	Bob	MATH201
3	Carol	PHY301

What you WANT: Join on SID only

$$\text{Student} \bowtie_{\text{Student.SID=Takes.SID}} \text{Takes}$$

Student.SID	Student.Name	Major	Takes.Name	Course
1	Alice	CS	Alice	CS101
2	Bob	Math	Bob	MATH201
3	Alice	Physics	Carol	PHY301

What NATURAL JOIN gives: Joins on BOTH SID AND Name!

$$\text{Student} \bowtie \text{Takes}$$

SID	Name	Major	Course
1	Alice	CS	CS101
2	Bob	Math	MATH201

PROBLEM: SID=3 (Alice in Physics taking PHY301) is LOST because the Name doesn't match (Alice $\neq$ Carol)!

Lesson: Natural join matched on SID=3 AND Name='Alice', but Takes has SID=3 with Name='Carol', so no match! Always be explicit about join conditions when tables share multiple attributes.

3.6 UNION (∪) - Combine Rows from Compatible Tables

Union Operation

Notation: $R \cup S$
What it does: Combines all rows from both tables, removes duplicates
Think of it as: "All rows from R OR S"

Requirement: R and S must be **union-compatible**:

1. Same number of attributes
2. Corresponding attributes have same domains (types)

Union Example

Table A: CS_Students

Name	Age
Alice	20
Bob	22

Table B: Math_Students

Name	Age
Bob	22
Carol	21

Query: CS_Students ∪ Math_Students
Result: All students (duplicates removed)

Name	Age
Alice	20
Bob	22
Carol	21

Note: Bob appears only once even though in both tables!

Key Insights About Union

- **Binary operation** - Takes TWO relations
- **Commutative:** $R \cup S = S \cup R$
- **Associative:** $(R \cup S) \cup T = R \cup (S \cup T)$
- **Removes duplicates** - Output is a set
- **Union-compatibility required!** - Can't union incompatible tables

3.7 INTERSECTION (∩) - Common Rows

Intersection Operation

Notation: $R \cap S$
What it does: Keeps only rows that appear in BOTH tables
Think of it as: "Rows in R AND S"

Requirement: R and S must be **union-compatible**

Intersection Example

Table A: CS_Students

Name	Age
Alice	20
Bob	22

Table B: Math_Students

Name	Age
Bob	22
Carol	21

Query: CS_Students ∩ Math_Students
Result: Students in BOTH CS and Math

Name	Age
Bob	22

Only Bob is in both!

3.8 SET DIFFERENCE (−) - Rows in One But Not Other

Set Difference Operation

Notation: $R - S$
What it does: Keeps rows in R that are NOT in S
Think of it as: "Rows in R BUT NOT in S"

Requirement: R and S must be **union-compatible**
NOT Commutative! $R - S \neq S - R$

Set Difference Example

Table A: CS_Students

Name	Age
Alice	20
Bob	22

Table B: Math_Students

Name	Age
Bob	22
Carol	21

Query 1: CS_Students − Math_Students
Result: CS students who are NOT in Math

Name	Age
Alice	20

Query 2: Math_Students − CS_Students
Result: Math students who are NOT in CS

Name	Age
Carol	21

Different results! Order matters!

Critical: Set Operations Are NOT Always Commutative!

Commutative:

- $R \cup S = S \cup R$
- $R \cap S = S \cap R$

NOT Commutative:

- $R - S \neq S - R$

Always check: Which table comes first matters for difference!

4 Advanced Operations

4.1 RENAME (ρ) - Change Names

Rename Operation

Notation: $\rho_{X(A_1,A_2,\ldots)}(R)$
What it does: Renames relation and/or attributes
Why needed:

- Avoid name conflicts in joins
- Make intermediate results clearer
- Self-joins (joining table with itself)

Rename Example

Table: Student

ID	Name
1	Alice
2	Bob

Rename relation: $\rho_{\text{Person}}(\text{Student})$
Result: Table now called "Person" with same data

Rename attributes: $\rho_{\text{Student(StudentID,StudentName)}}(\text{Student})$

StudentID	StudentName
1	Alice
2	Bob

4.2 ASSIGNMENT ($\leftarrow$) - Store Intermediate Results

Assignment Operation

Notation: TempName $\leftarrow$ Expression
What it does: Saves result of expression with a name
Why useful:

- Break complex queries into steps
- Avoid repeating sub-expressions
- Make queries more readable

Assignment Example

Goal: Find names of CS students older than 20

Without assignment (one big expression):

$\Pi_{\text{Name}}(\sigma_{\text{Age}>20 \wedge \text{Dept}=\text{'CS'}}(\text{Student}))$

With assignment (step by step):
`CSStudents` $\leftarrow \sigma_{\text{Dept}=\text{'CS'}}(\text{Student})$
`OldCSStudents` $\leftarrow \sigma_{\text{Age}>20}(\text{CSStudents})$
`Result` $\leftarrow \Pi_{\text{Name}}(\text{OldCSStudents})$
Much clearer!

4.3 Outer Joins - Keeping Non-Matching Rows

Why Outer Joins?

Problem with regular join:
If a row doesn't match anything in the other table, it disappears!

Solution: Outer Joins
Keep non-matching rows and fill missing values with NULL

Types of Outer Joins

1. **Left Outer Join** ($\ltimes$): Keep all rows from left table
2. **Right Outer Join** ($\rtimes$): Keep all rows from right table
3. **Full Outer Join** ($\bowtie$): Keep all rows from both tables

Outer Join Examples

Table A: Student

SID	Name
1	Alice
2	Bob
3	Carol

Table B: Enrollment

SID	Course
1	CS101
2	CS102

Note: Carol (SID=3) has no enrollment!

Regular Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102

Carol disappeared!

Left Outer Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102
3	Carol	NULL

Carol kept with NULL course!

Right Outer Join Example

Scenario: What if we have enrollments without matching students?
Table A: Student

SID	Name
1	Alice
2	Bob

Table B: Enrollment

SID	Course
1	CS101
2	CS102
3	CS103
4	MATH201

Note: SID=3 and SID=4 enrolled in courses but not in Student table!

Regular Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102

Enrollments for SID=3 and SID=4 disappeared!

Right Outer Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102
3	NULL	CS103
4	NULL	MATH201

All enrollments kept! Missing student info filled with NULL.

Full Outer Join Example

Scenario: Keep ALL students AND ALL enrollments
Table A: Student

SID	Name
1	Alice
2	Bob
3	Carol

Table B: Enrollment

SID	Course
1	CS101
2	CS102
4	MATH201

Note: Carol (SID=3) has no enrollment, and SID=4 enrolled but not in Student table!

Regular Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102

Both Carol and the MATH201 enrollment disappeared!

Full Outer Join: Student $\bowtie$ Enrollment

SID	Name	Course
1	Alice	CS101
2	Bob	CS102
3	Carol	NULL
4	NULL	MATH201

ALL data preserved! Non-matching rows filled with NULL.
Key Insight: Full Outer Join = Left Outer Join + Right Outer Join

Outer Joins Summary

Quick Comparison:

Join Type	Keeps	When to Use
Inner Join ($\bowtie$)	Only matches	When you want exact matches only
Left Outer ($\ltimes$)	All left rows	Keep all from left table
Right Outer ($\rtimes$)	All right rows	Keep all from right table
Full Outer	All rows from both	Keep everything from both tables

Memory Tip:

- **Left:** Arrow points left ($\ltimes$) - keep left table complete
- **Right:** Arrow points right ($\rtimes$) - keep right table complete
- **Full:** Arrows both ways - keep both tables complete

5 Aggregation and Grouping

Why Aggregation and Grouping?

Problem: Sometimes you want summary statistics, not individual rows

Examples:

- How many students per semester?
- What's the average salary in each department?
- What's the total enrollment per course?

Solution: Group rows by attribute values, then apply aggregate functions

Aggregation and Grouping Operation

Notation: $\gamma_{L;F}(r)$

Components:

- **L:** List of attributes for grouping (group-by attributes)
- **F:** Aggregate function(s) to apply to each group
- **r:** The relation to operate on

How it works:

1. Tuples with the same values for attributes in L are grouped together
2. Aggregate function F is applied to each group
3. Result has one row per group

Typical Aggregate Functions:

- `count(*)` - Number of tuples in a group
- `sum(A)` - Sum of attribute A values in a group
- `min(A)` - Minimum value of attribute A in a group
- `max(A)` - Maximum value of attribute A in a group
- `avg(A)` - Average value of attribute A in a group

5.1 Example 1: Count Students per Semester

Basic Grouping with Count

Query: Determine the number of students per semester
Relational Algebra: $\gamma_{\text{semester;count}(*)}(\text{Student})$

Input Table: Student

Name	Semester	Age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Christian	5	20
David	1	18

Result:

Semester	count(*)
7	1
3	2
5	2
1	1

Explanation:

- Students are grouped by semester
- Each group gets one row in result
- count(*) counts rows in each group
- Semester 3 has 2 students (Anna, Bob)
- Semester 5 has 2 students (Cathy, Christian)

5.2 Example 2: Multiple Aggregate Functions

Grouping with Multiple Aggregates

Query: Number of students per semester AND minimum age in each group
Relational Algebra: $\gamma_{\text{semester;count}(*),\text{min(age)}}(\text{Student})$

Input Table: Student

Name	Semester	Age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Christian	5	20
David	1	18

Result:

Semester	count(*)	min(age)
7	1	24
3	2	19
5	2	20
1	1	18

Explanation:

- Each group shows count AND minimum age
- Semester 3: 2 students, youngest is 19 (Bob)
- Semester 5: 2 students, youngest is 20 (Christian)

5.3 Example 3: All Aggregate Functions

Using Different Aggregate Functions

Query: Show count, sum, min, max, and avg salary per department
Input: Instructor

Name	Dept	Salary
Alice	CS	5000
Bob	CS	6000
Carol	Math	5500
Dave	Math	6500
Eve	CS	5500

Query:

$$\gamma_{\text{Dept;count}(*),\text{sum(Salary)},\text{min(Salary)},\text{max(Salary)},\text{avg(Salary)}}(\text{Instructor})$$

Result:

Dept	count(*)	sum	min	max	avg
CS	3	16500	5000	6000	5500
Math	2	12000	5500	6500	6000

Breakdown:

- **CS Dept:** 3 instructors, total 16500, salaries range 5000-6000, average 5500
- **Math Dept:** 2 instructors, total 12000, salaries range 5500-6500, average 6000

5.4 Aggregation Without Grouping

Aggregate Over Entire Relation

What if you want aggregate over ALL rows, not groups?

Solution: Use empty grouping list!

Query: Total number of students
Notation: $\gamma_{;\text{count}(*)}(\text{Student})$
Input:

Name	Semester	Age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22

Result:

count(*)
4

Note: Empty L means treat entire relation as one group!

5.5 Combining with Other Operations

Aggregation with SELECT and PROJECT

Query: Average salary of CS instructors

Step-by-step approach:

1. First filter to CS instructors: $\sigma_{\text{Dept='CS'}}(\text{Instructor})$
2. Then aggregate: $\gamma_{;\text{avg(Salary)}}(\sigma_{\text{Dept='CS'}}(\text{Instructor}))$

Another example: Count enrollments per course for CS courses only

$$\gamma_{\text{CourseID;count}(*)}(\sigma_{\text{CourseID LIKE 'CS\%'}}(\text{Enrollment}))$$

Steps:

1. SELECT filters to CS courses
2. GROUP BY CourseID
3. COUNT enrollments per course

Key Insights About Aggregation

- **Output size:** Result has one row per unique combination of grouping attributes

- **Empty grouping:** No grouping attributes means one row for entire relation
- **Multiple functions:** Can apply several aggregates in one operation
- **Attribute requirements:** Aggregate functions need appropriate attribute types (e.g., sum needs numeric)
- **NULL handling:** Most aggregates ignore NULL values (except count(*))

Common Mistakes with Aggregation

Mistake 1: Trying to select non-grouped attributes

WRONG: $\gamma_{\text{Dept;count}(*)}(\text{Instructor})$ then trying to access Name
Problem: After grouping by Dept, individual names are lost!

Rule: Result only contains:

- Grouping attributes (L)
- Aggregated values (F)

Mistake 2: Wrong aggregate function for data type
WRONG: sum(Name) - can't sum strings!
Correct: sum, avg, min, max work on numeric/comparable values

6 Operation Commutativity and Ordering

CRITICAL: Order Matters!

Understanding when you can reorder operations is KEY to solving problems!

Some operations can be swapped (commutative), others CANNOT!

6.1 Commutative Operations

Operations That CAN Be Reordered

1. **Multiple SELECTs:**

$$\sigma_p(\sigma_q(R)) = \sigma_q(\sigma_p(R)) = \sigma_{p \wedge q}(R)$$

2. **Union:**

$$R \cup S = S \cup R$$

3. **Intersection:**

$$R \cap S = S \cap R$$

4. **Cartesian Product:**

$$R \times S = S \times R \text{ (same data, columns reordered)}$$

5. **Natural Join:**

$$R \bowtie S = S \bowtie R \text{ (same data, columns reordered)}$$

6.2 NON-Commutative Operations

Operations Where Order MATTERS

1. **Set Difference:**

$$R - S \neq S - R$$

Example: Students in CS but not Math $\neq$ Students in Math but not CS

2. **PROJECT then SELECT (sometimes):**

$$\sigma_p(\Pi_A(R)) \text{ might fail if } p \text{ uses attributes not in } A$$

3. **Theta Join:**

$$R \bowtie_\theta S = S \bowtie_\theta R \text{ (data same, but column order different)}$$

Be careful with asymmetric conditions!

Order Matters Example

Given: Student(ID, Name, Age, Major)

WRONG ORDER:
$\sigma_{\text{Age}>20}(\Pi_{\text{Name}}(\text{Student}))$
This FAILS! After PROJECT, Age column is gone!

CORRECT ORDER:
$\Pi_{\text{Name}}(\sigma_{\text{Age}>20}(\text{Student}))$
Filter first (when Age exists), then project!

7 Query Optimization Rules

Why Optimization Matters

Same query can be written many ways!
Some ways are MUCH faster than others.

Key Principle: Reduce intermediate result sizes as early as possible

7.1 Rule 1: Push SELECT Down

SELECT Pushdown

Rule: Do SELECT operations as early as possible
Why: Reduces size before expensive operations (joins, products)

Formula:

$$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S \text{ if } p \text{ only uses } R \text{ attributes}$$

SELECT Pushdown Example

Query: Find CS students with their enrollments

BAD (Filter after join):
$\sigma_{\text{Major='CS'}}(\text{Student} \bowtie \text{Enrollment})$
Join ALL students first (expensive!), then filter

GOOD (Filter before join):
$(\sigma_{\text{Major='CS'}}(\text{Student})) \bowtie \text{Enrollment}$
Filter to CS students first (small table), then join

Why better: If you have 10,000 students but only 100 CS students, you only join 100 instead of 10,000!

7.2 Rule 2: Push PROJECT Down

PROJECT Pushdown

Rule: Do PROJECT early to eliminate unnecessary attributes
Caution: Must keep attributes needed for later operations!

Formula:

$$\Pi_{A \cup B}(R \bowtie S) = \Pi_A(R) \bowtie \Pi_B(S)$$

if A and B include join attributes

7.3 Rule 3: Combine Multiple SELECTs

SELECT Combination

Rule: Combine multiple SELECT conditions into one

Formula:

$$\sigma_p(\sigma_q(R)) = \sigma_{p \wedge q}(R)$$

Benefit: Single pass through data instead of multiple passes

7.4 Rule 4: Use Smaller Relation First in Join

Join Ordering

Rule: When joining multiple relations, start with smallest

Example:
If joining 3 tables A (100 rows), B (1000 rows), C (10 rows):
GOOD: $(C \bowtie A) \bowtie B$ (join small tables first)
BAD: $(B \bowtie A) \bowtie C$ (join large tables first)

Intermediate results matter!

8 Common Query Patterns and Examples

8.1 Pattern 1: Filter and Project

Find Names of Old Students

Goal: Get names of students older than 21
Given: Student(ID, Name, Age, Major)

Solution:

$$\Pi_{\text{Name}}(\sigma_{\text{Age}>21}(\text{Student}))$$

Steps:

1. SELECT to filter: keep students with Age > 21
2. PROJECT to get only Name column

8.2 Pattern 2: Join Two Tables

Find Students with Their Courses

Goal: Get student names with their enrolled courses
Given:

- Student(SID, Name)
- Enrollment(SID, CourseID)

Solution:

$$\Pi_{\text{Name, CourseID}}(\text{Student} \bowtie \text{Enrollment})$$

Steps:

1. Natural JOIN on SID
2. PROJECT to get Name and CourseID

8.3 Pattern 3: Join Multiple Tables

Find Students with Course Titles

Goal: Get student names with full course titles
Given:

- Student(SID, Name)
- Enrollment(SID, CourseID)
- Course(CourseID, Title)

Solution:

$$\Pi_{\text{Name, Title}}((\text{Student} \bowtie \text{Enrollment}) \bowtie \text{Course})$$

Steps:

1. Join Student and Enrollment on SID
2. Join result with Course on CourseID
3. PROJECT to get Name and Title

8.4 Pattern 4: Union - Combine Results

Find All Students (CS or Math)

Goal: Get all students who are in CS or Math department
Given: Student(ID, Name, Dept)

Solution:

$$\sigma_{\text{Dept='CS'}}(\text{Student}) \cup \sigma_{\text{Dept='Math'}}(\text{Student})$$

Alternative (simpler):

$$\sigma_{\text{Dept='CS'} \vee \text{Dept='Math'}}(\text{Student})$$

8.5 Pattern 5: Intersection - Find Common

Find Students in Both CS101 and CS102

Goal: Students enrolled in BOTH courses
Given: Enrollment(SID, CourseID)

Solution:

$$\Pi_{\text{SID}}(\sigma_{\text{CourseID='CS101'}}(\text{Enrollment})) \cap$$
$$\Pi_{\text{SID}}(\sigma_{\text{CourseID='CS102'}}(\text{Enrollment}))$$

Steps:

1. Get SIDs of students in CS101
2. Get SIDs of students in CS102
3. INTERSECT to find students in both

8.6 Pattern 6: Difference - Find Missing

Find Students NOT Enrolled

Goal: Students who haven't enrolled in any course
Given:

- Student(SID, Name)
- Enrollment(SID, CourseID)

Solution:

$$\Pi_{\text{SID}}(\text{Student}) - \Pi_{\text{SID}}(\text{Enrollment})$$

Steps:

1. Get all student IDs
2. Get IDs of enrolled students
3. SUBTRACT to find students with no enrollments

9 Complete Worked Examples

9.1 Example 1: Simple Query

Find Old CS Students

Given Schema:
Student(ID, Name, Age, Major)

Sample Data:

ID	Name	Age	Major
1	Alice	20	CS
2	Bob	22	CS
3	Carol	21	Math
4	Dave	23	CS

Query: Find names of CS students older than 21

Solution:

$$\Pi_{\text{Name}}(\sigma_{\text{Major='CS'} \wedge \text{Age}>21}(\text{Student}))$$

Step-by-Step Execution:
Step 1: $\sigma_{\text{Major='CS'} \wedge \text{Age}>21}(\text{Student})$

ID	Name	Age	Major
2	Bob	22	CS
4	Dave	23	CS

Step 2: Π_{Name} of above:

Name
Bob
Dave

Final Answer: Bob and Dave

9.2 Example 2: Join Query

Find Students with Course Titles

Given Schema:
Student(SID, Name)
Enrollment(SID, CourseID)
Course(CourseID, Title)

Sample Data:
Student:

SID	Name
1	Alice
2	Bob
3	Carol

Enrollment:

SID	CourseID
1	CS101
1	CS102
2	CS101

Course:

CourseID	Title
CS101	Databases
CS102	Algorithms

Query: Find name and course titles for all enrollments

Solution:

$$\Pi_{\text{Name, Title}}((\text{Student} \bowtie \text{Enrollment}) \bowtie \text{Course})$$

Step-by-Step:
Step 1: Student $\bowtie$ Enrollment

SID	Name	CourseID
1	Alice	CS101
1	Alice	CS102
2	Bob	CS101

Step 2: Above $\bowtie$ Course

SID	Name	CourseID	Title
1	Alice	CS101	Databases
1	Alice	CS102	Algorithms
2	Bob	CS101	Databases

Step 3: $\Pi_{\text{Name, Title}}$

Name	Title
Alice	Databases
Alice	Algorithms
Bob	Databases

9.3 Example 3: Set Operations

Students in CS or Math but not Both

Given Schema:
Student(ID, Name, Major)

Sample Data:

ID	Name	Major
1	Alice	CS
2	Bob	Math
3	Carol	CS
4	Dave	Math
5	Eve	CS

Query A: All students in CS or Math
Solution A:

$$\Pi_{\text{Name}}(\sigma_{\text{Major='CS'}}(\text{Student})) \cup \Pi_{\text{Name}}(\sigma_{\text{Major='Math'}}(\text{Student}))$$

Result A: Alice, Bob, Carol, Dave, Eve (all of them)

Query B: Students in CS but not Math
Solution B:

$$\Pi_{\text{Name}}(\sigma_{\text{Major='CS'}}(\text{Student})) - \Pi_{\text{Name}}(\sigma_{\text{Major='Math'}}(\text{Student}))$$

Step 1: CS students: Alice, Carol, Eve
Step 2: Math students: Bob, Dave
Step 3: Difference: Alice, Carol, Eve (none were in Math)

Query C: Students in Math but not CS
Solution C:

$$\Pi_{\text{Name}}(\sigma_{\text{Major='Math'}}(\text{Student})) - \Pi_{\text{Name}}(\sigma_{\text{Major='CS'}}(\text{Student}))$$

Result C: Bob, Dave

Key Observation: B and C give different results! Order matters for difference!

10 Common Mistakes and How to Avoid Them

10.1 Mistake 1: Wrong Order of Operations

PROJECT Before SELECT with Needed Attributes

WRONG:

$$\sigma_{\text{Age}>20}(\Pi_{\text{Name}}(\text{Student}))$$

Problem: After PROJECT, Age column doesn't exist!

CORRECT:

$$\Pi_{\text{Name}}(\sigma_{\text{Age}>20}(\text{Student}))$$

Rule: Always SELECT first if you need attributes that won't be in final result

10.2 Mistake 2: Forgetting Duplicates in Project

PROJECT Removes Duplicates

Common Error: Expecting duplicate values in PROJECT result

Remember: Relations are SETS, so duplicates are automatically removed

Example:
If 3 students have Age=20, Π_{Age} will show 20 only ONCE!

10.3 Mistake 3: Confusing Difference Order

Set Difference is NOT Commutative

Common Error: Thinking $A - B = B - A$

Truth: $A - B \neq B - A$ in general!

Example:

- $A = \{1, 2, 3\}$, $B = \{2, 3, 4\}$
- $A - B = \{1\}$ (in A but not B)
- $B - A = \{4\}$ (in B but not A)
- Completely different!

10.4 Mistake 4: Natural Join with Wrong Attributes

Natural Join Can Be Dangerous

Problem: Natural join uses ALL common attributes

Example:
If Student(ID, Name) and Enrollment(ID, Name) both have ID and Name, natural join will join on BOTH!

This means it will only match rows where BOTH ID and Name are equal, which is probably not what you want!

Solution: Use theta join with explicit condition or rename attributes

10.5 Mistake 5: Forgetting Union-Compatibility

Union/Intersection/Difference Require Compatible Relations

Requirements for $\cup$, $\cap$, $-$:

1. Same number of attributes
2. Corresponding attributes have same types

Common Error:
Trying to union Student(ID, Name, Age) with Course(CID, Title)
This FAILS! Different number of columns and different types!

Fix: PROJECT to make schemas compatible first

11 Quick Reference: All Operations

Operation	Symbol	Description
SELECT	$\sigma_p(R)$	Filter rows by condition p
PROJECT	$\Pi_{A_1,\ldots}(R)$	Keep only specified columns, remove duplicates
UNION	$R \cup S$	All rows from R or S (union-compatible required)
INTERSECTION	$R \cap S$	Rows in both R and S (union-compatible required)
DIFFERENCE	$R - S$	Rows in R but not in S (union-compatible required)
CARTESIAN PRODUCT	$R \times S$	All combinations of rows from R and S
THETA JOIN	$R \bowtie_\theta S$	$\sigma_\theta(R \times S)$ - Join with condition
NATURAL JOIN	$R \bowtie S$	Join on equal values of common attributes
RENAME	$\rho_X(R)$	Rename relation to X
ASSIGNMENT	$X \leftarrow R$	Store result as X
AGGREGATION	$\gamma_{L;F}(R)$	Group by attributes L, apply aggregate function F

Operation	Commutative?	Associative?
SELECT (σ)	YES	YES
PROJECT (Π)	NO	NO
UNION ($\cup$)	YES	YES
INTERSECTION ($\cap$)	YES	YES
DIFFERENCE ($-$)	NO	NO
CARTESIAN PRODUCT ($\times$)	YES*	YES
NATURAL JOIN ($\bowtie$)	YES*	YES
* Column order may differ, but data is the same		

12 Tips for Solving Relational Algebra Problems

Problem-Solving Strategy

Step 1: Understand the Question

- What information do you need to find?
- What are the input relations?
- What should the output look like?

Step 2: Identify Needed Operations

- Need to filter rows? → SELECT
- Need specific columns? → PROJECT
- Need to combine tables? → JOIN
- Need to combine results? → UNION
- Need common elements? → INTERSECTION
- Need to exclude? → DIFFERENCE

Step 3: Plan the Order

- Do SELECTs early (reduce data size)
- Do JOINs with needed attributes
- Do PROJECTs last (to get final columns)

Step 4: Write the Expression

- Start from innermost operation
- Build outward
- Check each step makes sense

Step 5: Verify

- Trace through with sample data
- Check column names exist at each step
- Verify final output matches question

Mental Shortcuts

Quick Translations:

- "Find rows WHERE..." → SELECT (σ)

- "Show only columns..." $\rightarrow$ PROJECT (Π)
- "Students AND their courses..." $\rightarrow$ JOIN ($\bowtie$)
- "Students OR teachers..." $\rightarrow$ UNION ($\cup$)
- "In both A AND B..." $\rightarrow$ INTERSECTION ($\cap$)
- "In A but NOT in B..." $\rightarrow$ DIFFERENCE ($-$)

13 Practice Problems

13.1 Problem Set

Practice Schemas

Use these schemas for the practice problems:
Student(SID, Name, Age, Major)
Course(CID, Title, Credits)
Enrollment(SID, CID, Grade)
Instructor(IID, Name, Dept)
Teaches(IID, CID, Semester)

Practice Problems:

1. Find names of all CS students
2. Find titles of all 3-credit courses
3. Find names of students enrolled in CS101
4. Find names of students enrolled in any course
5. Find names of students NOT enrolled in any course
6. Find names of students enrolled in both CS101 and CS102
7. Find names of instructors teaching CS courses
8. Find student names with their course titles
9. Find CS students older than 20 with their course titles
10. Find names of students who got grade 'A' in any course

Try these yourself before looking at solutions!
Hint: Start simple - what's the first operation you need? Then build from there.

13.2 Solutions

Solution 1: CS Students

Query: Find names of all CS students

$$\Pi_{\text{Name}}(\sigma_{\text{Major='CS'}}(\text{Student}))$$

Filter to CS, then get names.

Solution 2: 3-Credit Courses

Query: Find titles of all 3-credit courses

$$\Pi_{\text{Title}}(\sigma_{\text{Credits}=3}(\text{Course}))$$

Filter to 3 credits, then get titles.

Solution 3: Students in CS101

Query: Find names of students enrolled in CS101

$$\Pi_{\text{Name}}(\text{Student} \bowtie (\sigma_{\text{CID='CS101'}}(\text{Enrollment})))$$

Filter enrollments to CS101, join with Student, get names.

Solution 4: Students in Any Course

Query: Find names of students enrolled in any course

$$\Pi_{\text{Name}}(\text{Student} \bowtie \text{Enrollment})$$

Join to get enrolled students, project names (automatically removes duplicates).

Solution 5: Students NOT Enrolled

Query: Find names of students NOT enrolled in any course

$$\Pi_{\text{SID}}(\text{Student}) - \Pi_{\text{SID}}(\text{Enrollment})$$

All student IDs minus enrolled student IDs gives non-enrolled students.
To get names:

$$\Pi_{\text{Name}}(\text{Student} \bowtie (\Pi_{\text{SID}}(\text{Student}) - \Pi_{\text{SID}}(\text{Enrollment})))$$

Solution 6: Students in Both CS101 and CS102

Query: Students enrolled in both CS101 and CS102

$$\Pi_{\text{SID}}(\sigma_{\text{CID='CS101'}}(\text{Enrollment}))\cap$$
$$\Pi_{\text{SID}}(\sigma_{\text{CID='CS102'}}(\text{Enrollment}))$$

Get students in CS101, get students in CS102, intersect to find students in BOTH.

Solution 7: Instructors Teaching CS Courses

Query: Names of instructors teaching CS courses

$$\Pi_{\text{Name}}(\text{Instructor} \bowtie \text{Teaches} \bowtie (\sigma_{\text{CID LIKE 'CS\%'}}(\text{Course})))$$

Filter courses to CS courses, join with Teaches to get instructor IDs, join with Instructor to get names.
Note: This assumes course IDs start with 'CS' for CS courses

Solution 8: Students with Course Titles

Query: Student names with their course titles

$$\Pi_{\text{Name, Title}}((\text{Student} \bowtie \text{Enrollment}) \bowtie \text{Course})$$

Join Student with Enrollment (on SID), then join with Course (on CID), project to get Name and Title.

Solution 9: Old CS Students with Courses

Query: CS students older than 20 with their course titles

$$\Pi_{\text{Name, Title}}((\sigma_{\text{Major='CS'} \wedge \text{Age}>20}(\text{Student}) \bowtie \text{Enrollment}) \bowtie \text{Course})$$

Filter students first, then join with enrollments and courses, finally project needed columns.

Solution 10: Students with Grade A

Query: Names of students who got grade 'A'

$$\Pi_{\text{Name}}(\text{Student} \bowtie (\sigma_{\text{Grade='A'}}(\text{Enrollment})))$$

Filter enrollments to grade 'A', join with Student, get names.

14 Final Tips for Success

Key Takeaways

1. **Master the basics:** SELECT filters rows, PROJECT selects columns
2. **Order matters:** Always check if needed attributes exist at each step
3. **Optimize:** Push SELECT and PROJECT down (do early)
4. **Visualize:** Draw small sample tables and trace through operations
5. **Check commutativity:** Some operations can swap, others can't
6. **Use assignment:** Break complex queries into steps
7. **Practice:** The more problems you solve, the more patterns you'll recognize

Study Strategy

To truly master relational algebra:

1. Work through examples with pen and paper
2. Create your own small tables and trace operations
3. Try to solve problems before looking at solutions
4. Focus on understanding WHY each operation is used
5. Practice translating English queries to RA expressions

Remember

Relational Algebra is a Language
Like any language:

- You need to learn vocabulary (the operations)
- You need to learn grammar (how operations combine)
- You need to practice speaking it (solving problems)
- Eventually it becomes natural!

You've got this! Keep practicing and you'll master it.

Good luck with your studies!

Remember: Understanding beats memorization.

Think through each step, and it will all make sense.

15 Gør vi det

Vigtige Pointer

Husk disse nøglepunkter når du arbejder med relationel algebra:

1. **Forstå problemet først** - Læs opgaven grundigt og identificer hvad der søges
2. **Identificer de nødvendige tabeller** - Hvilke relationer skal bruges?
3. **Planlæg operationsrækkefølgen** - SELECT tidligt, PROJECT sent
4. **Verificer hvert trin** - Sørg for at attributter eksisterer når du bruger dem

Tjekliste til Eksamen

textbfFør du afleverer, tjek:

- ☐ Er alle attributnavne korrekte?
- ☐ Er operationsrækkefølgen logisk?
- ☐ Har du husket at PROJECT fjerner dubletter?
- ☐ Er dine JOIN-betingelser korrekte?
- ☐ Har du tjekket union-kompatibilitet for $\cup$, $\cap$, $-$?
- ☐ Er din notation konsistent og læselig?

Du kan det her!

Relationel algebra handler
om logisk tænkning

✓ Forstå de grundlæggende operationer
✓ Kan kombinere dem fornuftigt
✓ Øver dig på eksempler

Held og lykke!

Figure 1: Gør vi det!